

DEMI-JOURNÉE 2 / 6 - JOUR 1, APRÈS-MIDI

L'atelier du développeur : Visual Studio, modèles et Azure DevOps

S'équiper, puis réaliser sa première extension : le fil rouge ARC Delivery démarre

Date	Mardi 25 août 2026, après-midi (Slot 1 : 13h45-15h15, pause, Slot 2 : 15h30-17h00)
Format	Slot 1 (1 h 30) + pause 15 min + Slot 2 (1 h 30)
Public	Experts on-premises (AX 2009 à AX 2012 R3 / D365 on-prem)
Référence	MB-500 : Apply developer tools (5-10 %) + Design and develop AOT elements - Decks officiels 02 (modules 1-4) et 03 (module 2)



Rodrigue YENO
ARCHIA365 | ARCHIALEARN

Visual Studio, modèles et contrôle de source

Objectifs du slot

- Traduire ses réflexes MorphX dans Visual Studio et Application Explorer
- Créer un modèle d'extension et un projet, comprendre les dépendances
- Compiler, synchroniser la base et déboguer
- Versionner son code dans Azure DevOps (check-in, branches)
- Comprendre la chaîne de build et le deployable package

Déroulé (1 h 30)

- | | |
|--------|--|
| 20 min | De MorphX à Visual Studio : le mapping complet |
| 20 min | Modèles, packages, projets et dépendances |
| 15 min | Compilation, DB sync, débogage |
| 15 min | Azure DevOps : source control et build |
| 20 min | Démo D03 : premier modèle, premier code |

PARALLÈLE ON-PREM

Tout ce que vous faisiez dans MorphX se fait encore : autrement, ailleurs, et sous contrôle de source obligatoire.

Le mapping MorphX > Visual Studio

- Application Explorer = l'AOT en lecture : naviguer, chercher, examiner le standard
- Pour créer ou modifier : ajouter l'élément à un PROJET Visual Studio (dans votre modèle)
- Element designers : concepteurs graphiques par type d'élément (table, form, query...)
- X++ moderne : IntelliSense, refactoring, navigation par références
- Le Job AX disparaît : remplacé par la Runnable Class (méthode main)
 - Même usage : script ponctuel, test rapide, reprise de données

Vos réflexes traduits

AX / MorphX	D365 F&O Cloud
AOT (éditable)	Application Explorer (lecture)
Projet MorphX	Projet Visual Studio
Couche USR/CUS/VAR	Modèle d'extension
Job	Runnable class (main)
Compilation AOT	Build du projet / module
Debugger MorphX	Débogueur Visual Studio

PARALLÈLE ON-PREM

Le clic droit « Ouvrir en modification » sur une table standard n'existe plus : la lecture seule du standard n'est pas une permission manquante, c'est le principe fondateur du modèle Cloud.

Créer et gérer un modèle d'extension

- Assistant Create model (Visual Studio) : nom, éditeur, layer... et surtout le package
- **Choix structurant : « Create new package » (nouveau code) : la norme pour vos extensions**
- Références de modèles : déclarer les packages standard utilisés (ApplicationSuite, ApplicationPlatform...)
- Bonnes pratiques de nommage : préfixe société (ARC...) sur modèle et éléments
- Un modèle peut contenir extensions ET nouveaux objets ; il ne modifie JAMAIS le standard

Les pièges classiques

Sélectionner un package existant Microsoft

Ne mettez jamais votre code dans ApplicationSuite : créez votre propre package.

Références manquantes

Erreur de compilation « type introuvable » : ajoutez la référence du package qui contient l'objet standard visé.

Nommage sans préfixe

Sans préfixe ARC, collision possible avec le standard ou un ISV à la prochaine mise à jour.

PARALLÈLE ON-PREM

Le modèle joue le rôle de la couche (USR, CUS, VAR) et le package celui de la model store : mais contrairement aux couches, deux modèles ne s'écrasent pas l'un l'autre : ils coexistent et s'étendent.

Compiler, synchroniser, déboguer

- Build du projet : compilation X++ vers .NET (fini le p-code interprété)
- Les avertissements Best Practices sont intégrés au build : les traiter, pas les ignorer
- Synchronize database : applique le dictionnaire de données au SQL local du Tier-1
 - L'équivalent du « Synchronize » AX, mais déclenché explicitement
- Débogage : points d'arrêt, espions, pile d'appels dans Visual Studio
 - Déboguer le code standard : possible en chargeant les symboles, sans le modifier

Cycle de développement quotidien

- 1 Coder dans le projet (modèle ARC)
- 2 Build : compiler et corriger les BP
- 3 DB Sync si le modèle de données a changé
- 4 Exécuter / déboguer dans le client web
- 5 Check-in Azure DevOps en fin de tâche

PARALLÈLE ON-PREM

La compilation AX 2012 complète pouvait durer des heures et se faisait sur le serveur ; ici le build incrémental du modèle prend quelques minutes sur le Tier-1, et le build complet est délégué à la machine de build.

Azure DevOps : le contrôle de source n'est plus optionnel

- Connexion du Tier-1 à un projet Azure DevOps : mapping du dossier Metadata
- Cycle : Get latest > Check-out > modifications > Check-in (avec commentaire et lien work item)
- Branches : Main / Release, stratégie simple recommandée pour une équipe F&O
- Build pipeline : à chaque check-in (ou planifié), compilation complète + deployable package
- **Le package buildé rejoint l'Asset Library LCS : la boucle DJ1 est bouclée**

Pourquoi c'est non négociable :

La machine de build ne compile QUE ce qui est dans le contrôle de source.

Un fichier oublié sur le Tier-1 d'un développeur n'existe pas : il ne sera ni buildé, ni livré, ni en production.

Le contrôle de source est donc la définition même de « ce qui existe » dans le projet.

PARALLÈLE ON-PREM

MorphX VCS ou TFS étaient possibles en AX 2012 mais rarement déployés : le transfert se faisait par XPO de main en main. Ce mode artisanal disparaît : plus de « dernière version chez qui ? » : la vérité est dans DevOps.

Premier modèle, premier projet, premier code

Objectif

Créer le modèle ARCDelivery et son package, créer un projet Visual Studio, écrire une runnable class de test, compiler, exécuter, puis effectuer le premier check-in Azure DevOps.

Étapes clés

- Créer le modèle ARCDelivery (nouveau package)
- Créer le projet VS et le définir par défaut
- Écrire ARCHelloRunnable (main + info)
- Build, exécution dans le client web
- Check-in : le code entre dans l'histoire du projet

Environnement

Environnement de développement Tier-1 (cloud-hosted LCS ou UDE via PPAC) avec Visual Studio. À défaut : suivre sur la projection, le guide D03 permet de refaire chaque étape plus tard.

Durée indicative

20 minutes

Guide pas à pas

`demos/D03_modele_projet_runnable.md`

PAUSE

15 minutes

À la reprise : Le modèle d'extension : données et interface utilisateur (EDT, tables, entités, formulaires)

Le modèle d'extension : données et interface

Objectifs du slot

- Créer EDT, enums et tables dans son modèle : le geste inchangé
- Étendre une table standard : champs, groupes de champs, index, relations
- Comprendre le rôle central des data entities (import/export, OData, intégration)
- Étendre un formulaire standard : contrôles, sources de données, patterns
- Réaliser l'extension fil rouge : le champ Priorité de livraison sur le client

Déroulé (1 h 30)

15 min	EDT, enums, tables : créer du neuf
20 min	Étendre les tables standard
20 min	Data entities : le hub de données
15 min	Formulaires : patterns et extensions
20 min	Démo D04 : l'extension ARC Delivery

PARALLÈLE ON-PREM

La sur-couche répondait à « comment je modifie CustTable ? ». L'extension répond à « comment j'ajoute à CustTable sans y toucher ? ». Même besoin, mécanique inversée.

EDT, énumérations, tables : le geste presque inchangé

- EDT : typage métier réutilisable (ARCDeliveryPriorityId...), propriétés et héritage d'EDT
- Base enums : valeurs fixes métier ; extensibles si le standard le permet
 - Une enum standard extensible reçoit vos valeurs par extension d'enum
- Tables : champs, field groups, index, relations : le concepteur ressemble à l'AOT
- Propriétés à soigner dès la création : Label, Help text, Table group, Cache lookup
- Vues, queries et maps : mêmes concepts qu'en AX, mêmes usages

Fil rouge : les objets créés

Objet	Rôle
ARCDeliveryPriority (enum)	Basse / Normale / Haute / Critique
ARCDeliveryPriorityId (EDT)	Typage du champ, label unique
Champ sur CustTable (ext.)	Priorité portée par le client
Field group ARC (ext.)	Affichage groupé du champ
Entité (slot suivant)	Exposition du champ en OData

PARALLÈLE ON-PREM

Créer une table en 2026 ressemble à s'y méprendre à 2012 : les propriétés, index et relations sont les mêmes concepts. La différence est ADMINISTRATIVE : l'objet vit dans votre modèle, sous contrôle de source.

Étendre une table standard sans la toucher

- Clic droit sur la table dans Application Explorer > Create extension (dans votre modèle)
- L'extension .Extension contient : vos champs, field groups, index, relations
- Vos champs sont préfixés (ARC...) : lisibilité et absence de collision garanties
- **Ce qu'une extension de table NE peut PAS faire :**
 - Modifier un champ standard (type, taille) ni supprimer quoi que ce soit du standard
 - La logique (validation, valeurs par défaut) vit dans des classes d'extension : demain matin

Comparaison des mécaniques

AX 2012 : sur-couche

CustTable est copiée dans USR ; votre copie masque la SYS ; la montée de version exige la fusion manuelle des deux.

Cloud : extension

CustTable reste intacte ; CustTable.ARCExtension vit à côté ; la plateforme fusionne à l'exécution ; la mise à jour ne casse rien.

Bilan

Ce que vous perdez en liberté immédiate, vous le gagnez en pérennité à chaque mise à jour One Version.

PARALLÈLE ON-PREM

Le geste AX « j'ajoute mon champ directement dans CustTable » devient « je crée CustTable.ARCExtension avec mon champ » : cinq minutes de travail, zéro conflit de montée de version à vie.

Data entities : la porte d'entrée et de sortie des données

- Une entité = vue métier dénormalisée au-dessus des tables (Customers, Vendors...)
- Un objet, quatre usages : import/export (DMF), OData (API REST), intégration Excel, BYOD
- Staging + target : l'import transite par une table de staging (contrôle, reprise d'erreurs)
- Créer une entité : assistant depuis la table ; étendre une entité standard : extension d'entité
- Propriétés clés : Public collection name (OData), Data management enabled, champs calculés

Le champ ARC jusqu'à l'API

- 1 Champ ARCDeliveryPriority sur CustTable (extension)
- 2 Extension de l'entité CustCustomerV3 : ajout du champ
- 3 Build + DB sync : staging régénéré
- 4 Le champ apparaît dans Data management (import/export)
- 5 Le champ est requêtable en OData (démonstration DJ3)

PARALLÈLE ON-PREM

Les entités remplacent à la fois DIXF (AX 2012 R2/R3), une bonne part d'AIF, et les accès SQL directs des intégrations maison : un seul concept là où AX en empilait trois générations.

Formulaires : patterns imposés, extensions maîtrisées

- Chaque formulaire applique un PATTERN (Details Master, Simple List...) : structure normée et responsive
- Le pattern remplace le placement libre de contrôles d'AX : cohérence UX sur tout le produit
- Form extension : ajouter contrôles, sources de données, réordonner : sans toucher au standard
- Menus et menu items : mêmes concepts qu'AX (display/output/action), extensibles aussi
- Labels : fichiers de labels par modèle, multilingue dès la conception

Réflexe à désapprendre :

En AX, l'ergonomie d'un écran se sculptait contrôle par contrôle, pixel par pixel.

En Cloud, le pattern impose la structure : vous placez votre champ dans le bon groupe, la plateforme gère la disposition, le responsive et la cohérence.

Moins de liberté, moins de maintenance, plus d'homogénéité.

PARALLÈLE ON-PREM

Vos réflexes formulaires AX (data sources, groupes, grilles, méthodes de form) existent toujours : la nouveauté est le pattern obligatoire et le fait que toute retouche du standard passe par une form extension.

ARC Delivery : le champ Priorité visible dans l'écran client

Objectif

Créer l'enum et l'EDT de priorité, étendre CustTable (champ + field group), étendre le formulaire client pour afficher le champ, builder, synchroniser et vérifier le résultat dans le client web.

Étapes clés

- Créer ARCDeliveryPriority (enum) et l'EDT associé
- Extension de CustTable : champ + field group
- Extension du formulaire CustTable : ajout du champ
- Build + Synchronize database
- Vérification dans le client web + check-in

Environnement

Environnement de développement Tier-1 avec Visual Studio (modèle ARCDelivery créé en D03).

Durée indicative

20 minutes

Guide pas à pas

`demos/D04_extension_table_formulaire.md`

DEMI-JOURNÉE 2 - RÉCAPITULATIF

Ce qu'il faut retenir

- Application Explorer = AOT en lecture ; l'écriture passe par projet + modèle
- Modèle d'extension : votre espace, votre package, votre préfixe : le standard reste intact
- Le contrôle de source Azure DevOps définit ce qui existe : pas de check-in, pas de livraison
- Extension de table : ajouter sans modifier ; la logique viendra par les classes d'extension
- Data entities : un concept unique pour DMF, OData, Excel et l'analytique
- Fil rouge ARC Delivery : enum + EDT + champ + formulaire opérationnels et versionnés

Prochaine étape

DJ3 (demain matin) : X++ moderne, Chain of Command, événements, frameworks (SysOperation) et tests : le champ ARC reçoit sa logique métier.